Milestone 2 — Classical ML baseline for skin-lesion classification (MILK10k)

Course: Computer Vision and Speech Recognition — EADA **Student:** Kristof Papp

Code (GitHub): https://github.com/kristofpapp330/milk10k-cv-course/tree/main/notebooks

what	file in the repo
this notebook (experiments + report)	`notebooks/classical_baseline.ipynb`
feature extraction	`milk10k_pipeline/features.py`
models, model selection, metrics	`milk10k_pipeline/classical.py`
tests (incl. the Session 5 hand calculations)	`tests/test_features.py`
splits re-used from Milestone 1	`outputs/splits/`
all results tables	`outputs/baseline/`

0. Task and plan

Task (project). From the **dermoscopic image** of a skin lesion, predict **Benign** vs **Malignant** (`diagnosis_1` , with the 123 *Indeterminate* lesions merged into *Malignant* = "needs clinical action", as decided in Milestone 1). As an extra, the same pipeline is also run on the **3 original classes** (Benign / Malignant / Indeterminate), as in the Session 5 slides.

Why a classical baseline first (Session 5). It tells us (1) whether simple colour and texture signals predict the diagnosis at all, (2) whether the Milestone 1 pipeline works end to end, and (3) gives the number that the CNN of Session 7 must beat.

Plan — the Session 5 baseline recipe

1. **Split:** the Milestone 1 lesion-level split (60/20/20 %, grouped by `lesion_id`, stratified on the 11 classes, seed 42), re-used unchanged. One dermoscopic image per lesion.
2. **Features:** handcrafted colour, histogram, texture, GLCM and ABCD-style features, grouped into **5 feature sets**.
3. **Models:** majority class (the floor), logistic regression, kNN, random forest, SVM (RBF kernel), each with a small hyper-parameter grid.
4. **Model selection on VALIDATION** with **macro-F1** (accuracy is misleading with ~70 % malignant).
5. **TEST once** with the chosen configuration: metrics with 95 % bootstrap intervals, confusion matrix, ROC curve, errors per diagnosis.

How to run: *Restart & Run All*. The data folder is set in `milk10k_pipeline/config.py` (or the environment variable `MILK10K_DIR`). The first run extracts the features from the 5,240 images (about 5–15 minutes) and saves them to `outputs/baseline/features_dermoscopic.csv`; later runs load that file and take about 2–4 minutes.

1. Setup and data

In [1]:
```
import json
import sys
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.inspection import permutation_importance
from sklearn.metrics import roc_auc_score, roc_curve

warnings.filterwarnings("ignore")                # keep the PDF readable

REPO = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(REPO))                    # make the project package importable

from milk10k_pipeline import classical, config, features
from milk10k_pipeline.data_io import find_images, load_rgb

BASE_DIR = config.OUTPUT_DIR / "baseline"        # all tables of this milestone
BASE_DIR.mkdir(parents=True, exist_ok=True)
config.FIGURES_DIR.mkdir(parents=True, exist_ok=True)
FEATURE_CACHE = BASE_DIR / "features_dermoscopic.csv"
```

```
RECOMPUTE_FEATURES = False                          # True = ignore the cache and extract again

# colours: one fixed colour per class, used in every figure
COLORS = {"Benign": "#2a78d6", "Malignant": "#eb6834", "Indeterminate": "#1baf7a"}
BINARY = ["Benign", "Malignant"]
THREE = ["Benign", "Malignant", "Indeterminate"]

def save(name):
    plt.savefig(config.FIGURES_DIR / f"baseline_{name}.png", dpi=130, bbox_inches="tight")

print("data folder:", config.DATA_DIR)
print("seed:", config.SEED)
```

```
data folder: C:\Users\krisi\milk10k-cv-course\data\milk10k
seed: 42
```

Load the Milestone 1 splits and keep **one dermoscopic image per lesion** (`config.VIEWS`). Labels: `label` = binary target (0 Benign, 1 Malignant incl. Indeterminate); `label3` = the 3 original `diagnosis_1` classes for the extra experiment.

In [2]:
```
splits = {}
for name in ["train", "val", "test"]:
    df = pd.read_csv(config.SPLITS_DIR / f"{name}.csv")
    df = df[df["view"].isin(config.VIEWS)].reset_index(drop=True)
    df["label3"] = df["diagnosis_1"].map({c: i for i, c in enumerate(THREE)})
    splits[name] = df
train, val, test = splits["train"], splits["val"], splits["test"]

# safety checks: one image per lesion, no lesion in two splits, labels as in Milestone 1
for name, df in splits.items():
    assert df["lesion_id"].is_unique, f"{name}: a lesion appears twice"
assert not (set(train.lesion_id) & set(val.lesion_id)) and not (set(train.lesion_id) & set(test.lesion_id)) \
    and not (set(val.lesion_id) & set(test.lesion_id)), "lesion overlap between splits!"
assert (train["label"] == (train["diagnosis_1"] != "Benign").astype(int)).all()

counts = pd.DataFrame({name: df["diagnosis_1"].value_counts() for name, df in splits.items()}).loc[THREE]
counts.loc["total"] = counts.sum()
counts.loc["% Malignant (binary)"] = [round(100 * df["label"].mean(), 1) for df in splits.values()]
counts
```

Out[2]:

diagnosis_1	train	val	test
Benign	889.0	296.0	298.0
Malignant	2182.0	724.0	728.0
Indeterminate	72.0	28.0	23.0
total	3143.0	1048.0	1049.0
% Malignant (binary)	71.7	71.8	71.6

2. Features

Every image is resized to 256 × 256 and described by **45 numbers** in 5 groups (code: `milk10k_pipeline/features.py`):

group	n	features	idea
colour statistics	6	mean and std of R, G, B	overall colour / colour variability (Session 5, Session 2 finding)
colour histograms	24	8 bins × 3 channels, as fractions of the pixels	colour distribution (Session 5)
texture	3	edge density (mean Sobel magnitude), roughness (mean 9×9 local variance), Laplacian variance	Session 4 filters averaged into one number
GLCM	4	contrast, homogeneity, energy, correlation (32 grey levels, 4 directions)	co-occurrence texture (Session 5 "optional extension … good idea for your project")
ABCD	8	asymmetry left-right and top-bottom (2); from a rough lesion mask: area, compactness = perimeter^2/(4π·area), share of the image border covered, colour contrast skin − lesion in R, G, B (6)	dermatology checklist (Session 5 ABCDE slide)

The 5 feature sets compared: `color (30)` · `texture (3)` · `color+texture (33)` (= the Session 5 vector) · `color+texture+GLCM (37)` · `all incl. ABCD (45)` .

Correctness check. `tests/test_features.py` verifies that the code reproduces the hand calculations of the Session 5 slides on the 5×5 toy lesion: histogram [0.36, 0.64, 0, 0], mean 67.6, std 43.2, edge density 279.8.

In [3]:
```
image_paths = find_images()
all_images = pd.concat([train, val, test], ignore_index=True)
feats = features.extract_dataset(all_images, image_paths, cache_path=FEATURE_CACHE, recompute=RECOMPUTE_FEATURES)
print("feature table:", feats.shape)
print("images where the rough lesion mask failed (ABCD mask features = NaN, filled with the TRAIN median):",
      int(feats["mask_area"].isna().sum()))
feats.head(3).round(3)
```

```
 500/5240 images  (16s elapsed, ~147s left)
 1000/5240 images  (30s elapsed, ~125s left)
 1500/5240 images  (44s elapsed, ~109s left)
 2000/5240 images  (57s elapsed, ~93s left)
 2500/5240 images  (72s elapsed, ~78s left)
 3000/5240 images  (86s elapsed, ~64s left)
 3500/5240 images  (100s elapsed, ~50s left)
 4000/5240 images  (117s elapsed, ~36s left)
 4500/5240 images  (134s elapsed, ~22s left)
 5000/5240 images  (148s elapsed, ~7s left)
 5240/5240 images  (155s elapsed, ~0s left)
saved features to C:\Users\krisi\milk10k-cv-course\outputs\baseline\features_dermoscopic.csv
feature table: (5240, 45)
images where the rough lesion mask failed (ABCD mask features = NaN, filled with the TRAIN median): 695
```

Out[3]:

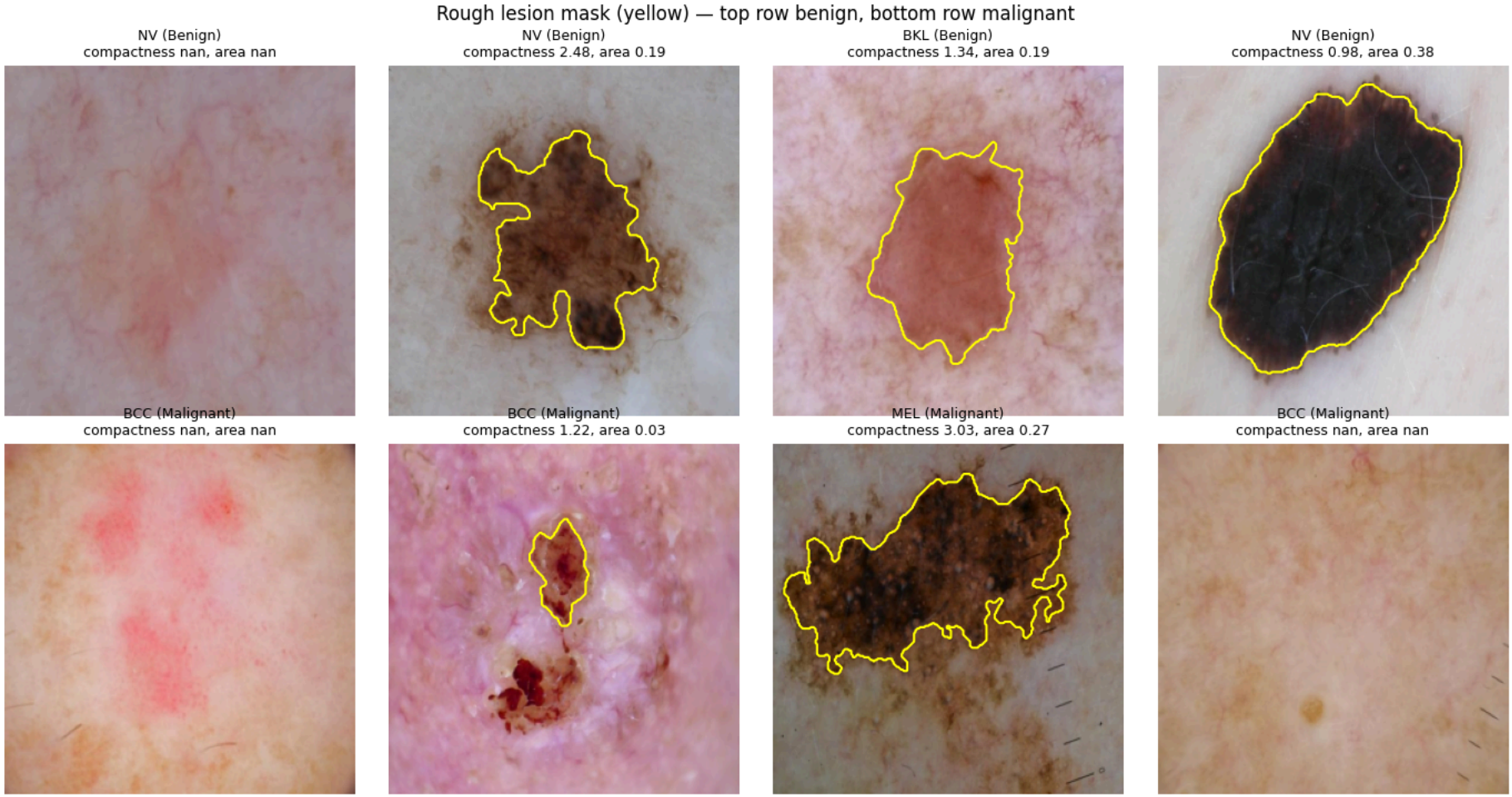
isic_id	mean_R	std_R	mean_G	std_G	mean_B	std_B	hist_R0	hist_R1	hist_R2	hist_R3	...	glcm_energy	glcm_correlation	as
ISIC_3624913	171.298	13.034	146.530	28.597	145.180	27.053	0.0	0.000	0.001	0.005	...	0.289	0.963	
ISIC_1048297	199.742	39.456	203.506	35.547	215.121	30.268	0.0	0.008	0.018	0.044	...	0.237	0.952	
ISIC_9546309	187.878	12.177	140.053	14.141	124.496	20.908	0.0	0.000	0.000	0.001	...	0.310	0.932	

3 rows × 45 columns

2.1 What the rough lesion mask sees

The ABCD features need a lesion mask. Real segmentation comes in Session 8, so here it is a simple approximation: Gaussian blur (σ = 2) → Otsu threshold → the dark region that overlaps the image centre. The figure shows it on random training images, to judge how reliable these features can be.

In [4]:
```
rng = np.random.default_rng(config.SEED)
show = pd.concat([train[train["label"] == k].sample(4, random_state=config.SEED) for k in [0, 1]])
fig, axes = plt.subplots(2, 4, figsize=(14, 7.4))
for ax, r in zip(axes.flat, show.itertuples()):
    rgb = features.load_for_features(image_paths[r.isic_id])
    mask = features.lesion_mask(features.to_gray(rgb.astype(float)))
    ax.imshow(rgb)
    if mask is not None:
        ax.contour(mask, levels=[0.5], colors="yellow", linewidths=1.5)
    f = feats.loc[r.isic_id]
    ax.set_title(f"{r.dx} ({r.diagnosis_1})\ncompactness {f['mask_compactness']:.2f}, area {f['mask_area']:.2f}",
                 fontsize=9)
    ax.axis("off")
plt.suptitle("Rough lesion mask (yellow) — top row benign, bottom row malignant")
plt.tight_layout(); save("mask_examples"); plt.show()
```



Observation. The rough mask works on pigmented (brown/black) lesions, but it fails on pale, pink, non-pigmented lesions, which are not darker than the skin: no mask for **695 of 5,240 images (13 %)**. Their mask features are filled with the train median. The compactness of irregular lesions is high (MEL 3.03, irregular NV 2.48) and that of round ones is about 1 (NV 0.98), as intended.

2.2 Is there ONE feature that separates the classes? (Session 5 question)

For each feature alone, the ROC-AUC on the **training** split (0.5 = useless; values below 0.5 mean the feature is *lower* for malignant lesions, so |AUC − 0.5| is the strength).

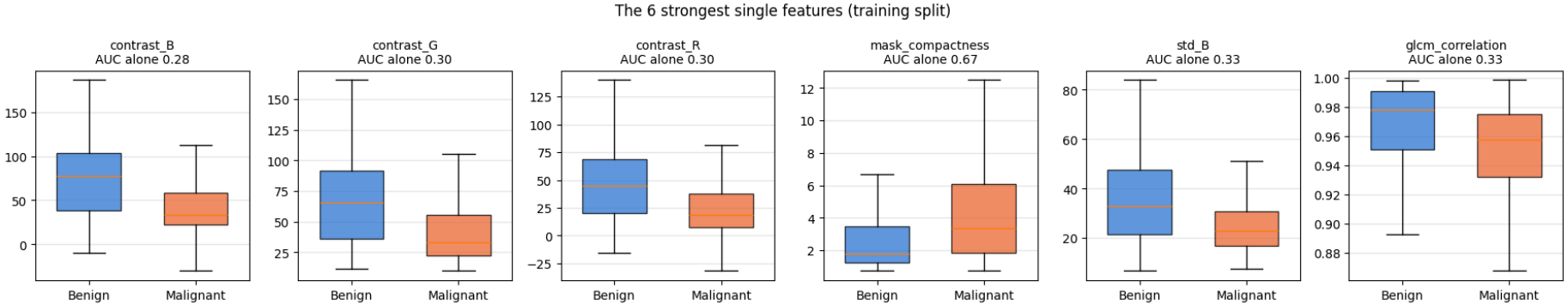
In [5]:
```
y_tr = train["label"].to_numpy()
F_tr = feats.loc[train.isic_id]
single = pd.DataFrame({"AUC alone": [roc_auc_score(y_tr, F_tr[c].fillna(F_tr[c].median())) for c in feats.columns]},
                      index=feats.columns)
single["strength |AUC-0.5|"] = (single["AUC alone"] - 0.5).abs()
single["higher in"] = np.where(single["AUC alone"] > 0.5, "Malignant", "Benign")
single = single.sort_values("strength |AUC-0.5|", ascending=False)
single.head(12).round(3)
```

Out[5]:

	AUC alone	strength |AUC-0.5|	higher in
contrast_B	0.280	0.220	Benign
contrast_G	0.297	0.203	Benign
contrast_R	0.305	0.195	Benign
mask_compactness	0.674	0.174	Malignant
std_B	0.327	0.173	Benign
glcm_correlation	0.331	0.169	Benign
std_G	0.348	0.152	Benign
hist_B4	0.643	0.143	Malignant
hist_B3	0.640	0.140	Malignant
hist_B1	0.361	0.139	Benign
mask_border	0.623	0.123	Malignant
hist_G1	0.386	0.114	Benign

In [6]:
```
top = single.index[:6]
fig, axes = plt.subplots(1, 6, figsize=(18, 3.6))
for ax, col in zip(axes, top):
    data = [feats.loc[train.isic_id[train.label == k], col].dropna() for k in [0, 1]]
    bp = ax.boxplot(data, widths=0.6, showfliers=False, patch_artist=True)
    for patch, cls in zip(bp["boxes"], BINARY):
        patch.set_facecolor(COLORS[cls]); patch.set_alpha(0.75)
```

```
    ax.set_xticks([1, 2], BINARY); ax.set_title(f"{col}\nAUC alone {single.loc[col, 'AUC alone']:.2f}", fontsize=10)
    ax.grid(axis="y", alpha=0.3)
plt.suptitle("The 6 strongest single features (training split)")
plt.tight_layout(); save("top_features"); plt.show()
```



Observation. No single feature separates the classes. The best ones reach an AUC of only about 0.70 (|AUC − 0.5| ≈ 0.2). The strongest are the **skin–lesion colour contrasts**, which are *lower* in malignant lesions. The reason is the class mix rather than melanoma biology: 67 % of the malignant lesions are **BCC**, often pink and barely darker than the skin. Most benign lesions are **nevi (NV)**, dark and high-contrast. Malignant lesions also have less round masks (higher compactness) and less blue variability (std_B).

3. Model comparison on VALIDATION (binary task)

Every model is a pipeline **median imputer → StandardScaler → classifier**; the imputer and the scaler are fitted on TRAIN only. `class_weight="balanced"` is used for logistic regression, random forest and SVM (kNN has no class weights). Hyper-parameter grids (`classical.PARAM_GRIDS`):

In [7]:
```
pd.DataFrame({"values tried": {m: str(g) if g else "—" for m, g in classical.PARAM_GRIDS.items()},
              "n configs": {m: len(classical.param_combinations(m)) for m in classical.PARAM_GRIDS}})
```

Out[7]:

	values tried	n configs
majority	—	1
logistic_reg	{'C': [0.01, 0.1, 1, 10]}	4
knn	{'n_neighbors': [5, 15, 31, 51], 'weights': ['...	8
random_forest	{'max_depth': [None, 12], 'min_samples_leaf': ...	4
svm_rbf	{'C': [0.3, 1, 3, 10]}	4

In [8]:
```
results = classical.run_grid(feats, train, val, "label", BINARY)
results.assign(params=results["params"].astype(str)).to_csv(BASE_DIR / "val_results_binary.csv", index=False)
print(f"\n{len(results)} configurations evaluated on validation")
```

```
color (30)                majority       best val macro-F1 = 0.418
color (30)                logistic_reg   best val macro-F1 = 0.671
color (30)                knn            best val macro-F1 = 0.673
color (30)                random_forest  best val macro-F1 = 0.717
color (30)                svm_rbf        best val macro-F1 = 0.724
texture (3)               majority       best val macro-F1 = 0.418
texture (3)               logistic_reg   best val macro-F1 = 0.516
texture (3)               knn            best val macro-F1 = 0.519
texture (3)               random_forest  best val macro-F1 = 0.562
texture (3)               svm_rbf        best val macro-F1 = 0.595
color+texture (33)        majority       best val macro-F1 = 0.418
color+texture (33)        logistic_reg   best val macro-F1 = 0.691
color+texture (33)        knn            best val macro-F1 = 0.676
color+texture (33)        random_forest  best val macro-F1 = 0.723
color+texture (33)        svm_rbf        best val macro-F1 = 0.736
color+texture+GLCM (37)   majority       best val macro-F1 = 0.418
color+texture+GLCM (37)   logistic_reg   best val macro-F1 = 0.713
color+texture+GLCM (37)   knn            best val macro-F1 = 0.684
color+texture+GLCM (37)   random_forest  best val macro-F1 = 0.723
color+texture+GLCM (37)   svm_rbf        best val macro-F1 = 0.732
all incl. ABCD (45)       majority       best val macro-F1 = 0.418
all incl. ABCD (45)       logistic_reg   best val macro-F1 = 0.726
all incl. ABCD (45)       knn            best val macro-F1 = 0.703
all incl. ABCD (45)       random_forest  best val macro-F1 = 0.739
all incl. ABCD (45)       svm_rbf        best val macro-F1 = 0.741

105 configurations evaluated on validation
```

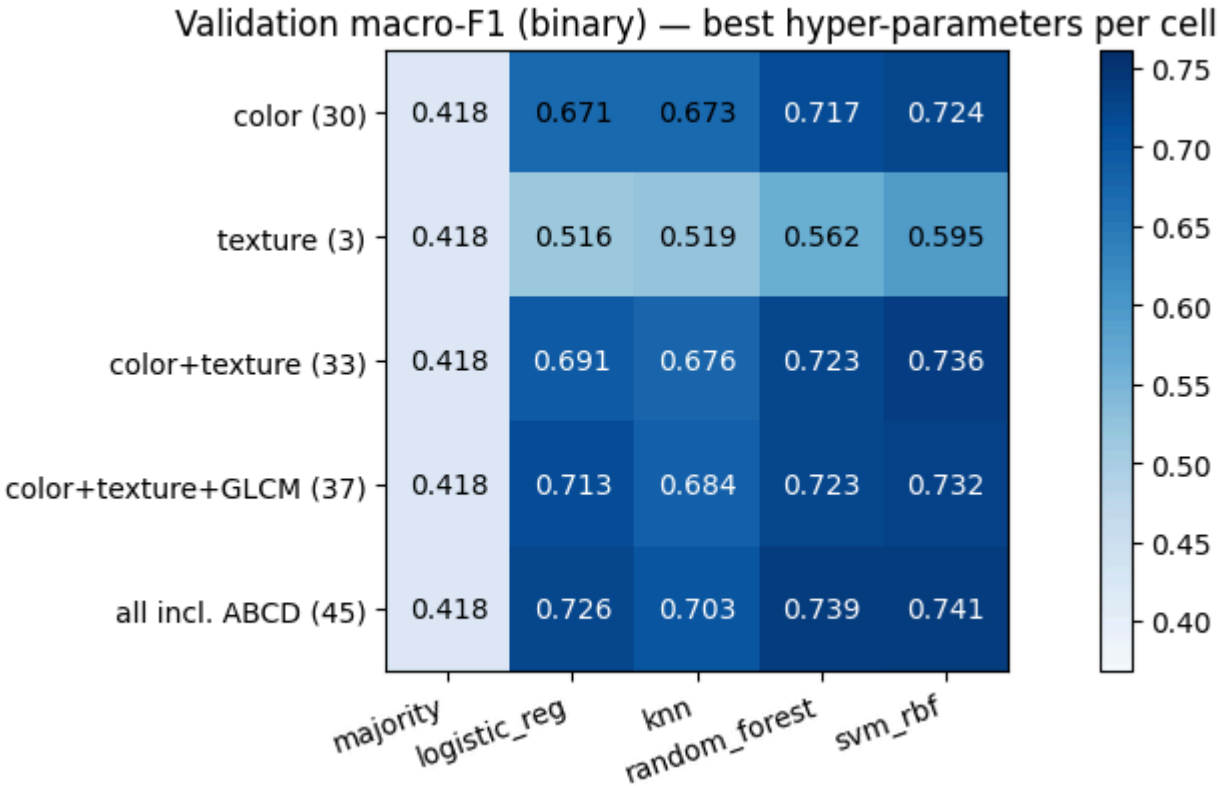
3.1 Best validation macro-F1 for every feature set × model

(each cell = the best hyper-parameters of that model on that feature set)

In [9]:
```
cells = classical.best_per_cell(results)
model_order = list(classical.PARAM_GRIDS)
set_order = list(features.FEATURE_SETS)
grid = cells.pivot(index="feature_set", columns="model", values="macro_f1").loc[set_order, model_order]

fig, ax = plt.subplots(figsize=(10, 4.2))
im = ax.imshow(grid.values, cmap="Blues", vmin=grid.values.min() - 0.05, vmax=grid.values.max() + 0.02)
```

```
ax.set_xticks(range(len(model_order)), model_order, rotation=20, ha="right")
ax.set_yticks(range(len(set_order)), set_order)
for i in range(len(set_order)):
    for j in range(len(model_order)):
        v = grid.values[i, j]
        ax.text(j, i, f"{v:.3f}", ha="center", va="center", fontsize=10,
                color="white" if v > grid.values.mean() + 0.05 else "black")
ax.set_title("Validation macro-F1 (binary) — best hyper-parameters per cell")
plt.colorbar(im, ax=ax, fraction=0.03); plt.tight_layout(); save("val_heatmap"); plt.show()
```



Observation.

- **Feature sets:** colour carries most of the signal (best 0.724). Texture alone is weak (0.595), but adds a little on top of colour (+0.01 with SVM, +0.02 with logistic regression). GLCM helps the linear model (+0.02) but not SVM or random forest. The ABCD features give the best result for every model, with gains of 0.01–0.02.

- **Models:** SVM (RBF) ≈ random forest > logistic regression > kNN, for every feature set. The non-linear models gain about 0.015–0.05 over the linear one. kNN is the weakest because all 45 features count equally in its distance, even the useless ones.
- Every real model is far above the **majority floor (0.418)**.

In [10]:
```
table = cells[["feature_set", "model", "params", "macro_f1", "balanced_acc", "roc_auc",
               "sensitivity", "specificity", "accuracy"]].copy()
table["params"] = table["params"].astype(str)
table.sort_values("macro_f1", ascending=False).round(3).reset_index(drop=True)
```

Out[10]:

	feature_set	model	params	macro_f1	balanced_acc	roc_auc	sensitivity	specificity	accuracy
0	all incl. ABCD (45)	svm_rbf	{'C': 3}	0.741	0.743	0.797	0.847	0.639	0.788
1	all incl. ABCD (45)	random_forest	{'max_depth': None, 'min_samples_leaf': 5}	0.739	0.722	0.797	0.916	0.527	0.806
2	color+texture (33)	svm_rbf	{'C': 1}	0.736	0.738	0.786	0.844	0.632	0.784
3	color+texture+GLCM (37)	svm_rbf	{'C': 3}	0.732	0.735	0.790	0.838	0.632	0.780
4	all incl. ABCD (45)	logistic_reg	{'C': 1}	0.726	0.737	0.792	0.811	0.662	0.769
5	color (30)	svm_rbf	{'C': 3}	0.724	0.724	0.774	0.846	0.601	0.777
6	color+texture+GLCM (37)	random_forest	{'max_depth': 12, 'min_samples_leaf': 5}	0.723	0.708	0.788	0.908	0.507	0.795
7	color+texture (33)	random_forest	{'max_depth': None, 'min_samples_leaf': 5}	0.723	0.708	0.784	0.903	0.514	0.793
8	color (30)	random_forest	{'max_depth': None, 'min_samples_leaf': 5}	0.717	0.702	0.777	0.904	0.500	0.790
9	color+texture+GLCM (37)	logistic_reg	{'C': 1}	0.713	0.719	0.778	0.819	0.618	0.762
10	all incl. ABCD (45)	knn	{'n_neighbors': 31, 'weights': 'distance'}	0.703	0.682	0.774	0.939	0.426	0.794
11	color+texture (33)	logistic_reg	{'C': 1}	0.691	0.704	0.765	0.777	0.632	0.736
12	color+texture+GLCM (37)	knn	{'n_neighbors': 15, 'weights': 'distance'}	0.684	0.664	0.761	0.947	0.382	0.787
13	color+texture (33)	knn	{'n_neighbors': 15, 'weights': 'distance'}	0.676	0.658	0.750	0.947	0.368	0.783
14	color (30)	knn	{'n_neighbors': 5, 'weights': 'distance'}	0.673	0.659	0.728	0.896	0.422	0.762
15	color (30)	logistic_reg	{'C': 0.01}	0.671	0.690	0.760	0.741	0.639	0.712

	feature_set	model	params	macro_f1	balanced_acc	roc_auc	sensitivity	specificity	accuracy
16	texture (3)	svm_rbf	{'C': 3}	0.595	0.602	0.620	0.723	0.480	0.655
17	texture (3)	random_forest	{'max_depth': 12, 'min_samples_leaf': 5}	0.562	0.564	0.613	0.726	0.402	0.635
18	texture (3)	knn	{'n_neighbors': 5, 'weights': 'distance'}	0.519	0.524	0.558	0.842	0.206	0.662
19	texture (3)	logistic_reg	{'C': 0.01}	0.516	0.556	0.557	0.507	0.605	0.534
20	color (30)	majority	{}	0.418	0.500	NaN	1.000	0.000	0.718
21	texture (3)	majority	{}	0.418	0.500	NaN	1.000	0.000	0.718
22	color+texture (33)	majority	{}	0.418	0.500	NaN	1.000	0.000	0.718
23	color+texture+GLCM (37)	majority	{}	0.418	0.500	NaN	1.000	0.000	0.718
24	all incl. ABCD (45)	majority	{}	0.418	0.500	NaN	1.000	0.000	0.718

3.2 The chosen configuration

Rule fixed in advance: **highest validation macro-F1** (ties: higher balanced accuracy).

In [11]:
```
best = classical.select_best(results)
print("CHOSEN:", best["feature_set"], "|", best["model"], "|", best["params"])
print(f"validation: macro-F1 {best['macro_f1']:.3f}, balanced acc {best['balanced_acc']:.3f}, "
      f"ROC-AUC {best['roc_auc']:.3f}, sensitivity {best['sensitivity']:.3f}, specificity {best['specificity']:.3f}")

floor = cells[cells["model"] == "majority"].iloc[0]
print(f"majority-class floor: macro-F1 {floor['macro_f1']:.3f}, accuracy {floor['accuracy']:.3f}")

model, cols = classical.fit_config(best, feats, train, "label")
X_va, y_va = feats.loc[val.isic_id, cols].to_numpy(), val["label"].to_numpy()
score_va = classical.scores_of(model, X_va)
```

```
CHOSEN: all incl. ABCD (45) | svm_rbf | {'C': 3}
validation: macro-F1 0.741, balanced acc 0.743, ROC-AUC 0.797, sensitivity 0.847, specificity 0.639
majority-class floor: macro-F1 0.418, accuracy 0.718
```

Observation. The winner is **all 45 features + SVM (RBF, C = 3)** with a validation macro-F1 of **0.741**. The margin over the next configurations is small: random forest 0.739, SVM on the 33 Session 5 features 0.736. With about 1,050 validation lesions, the uncertainty of one score is around ±0.03, so the top 4–5 configurations are practically tied. The selection rule was fixed in advance, so the winner is kept.

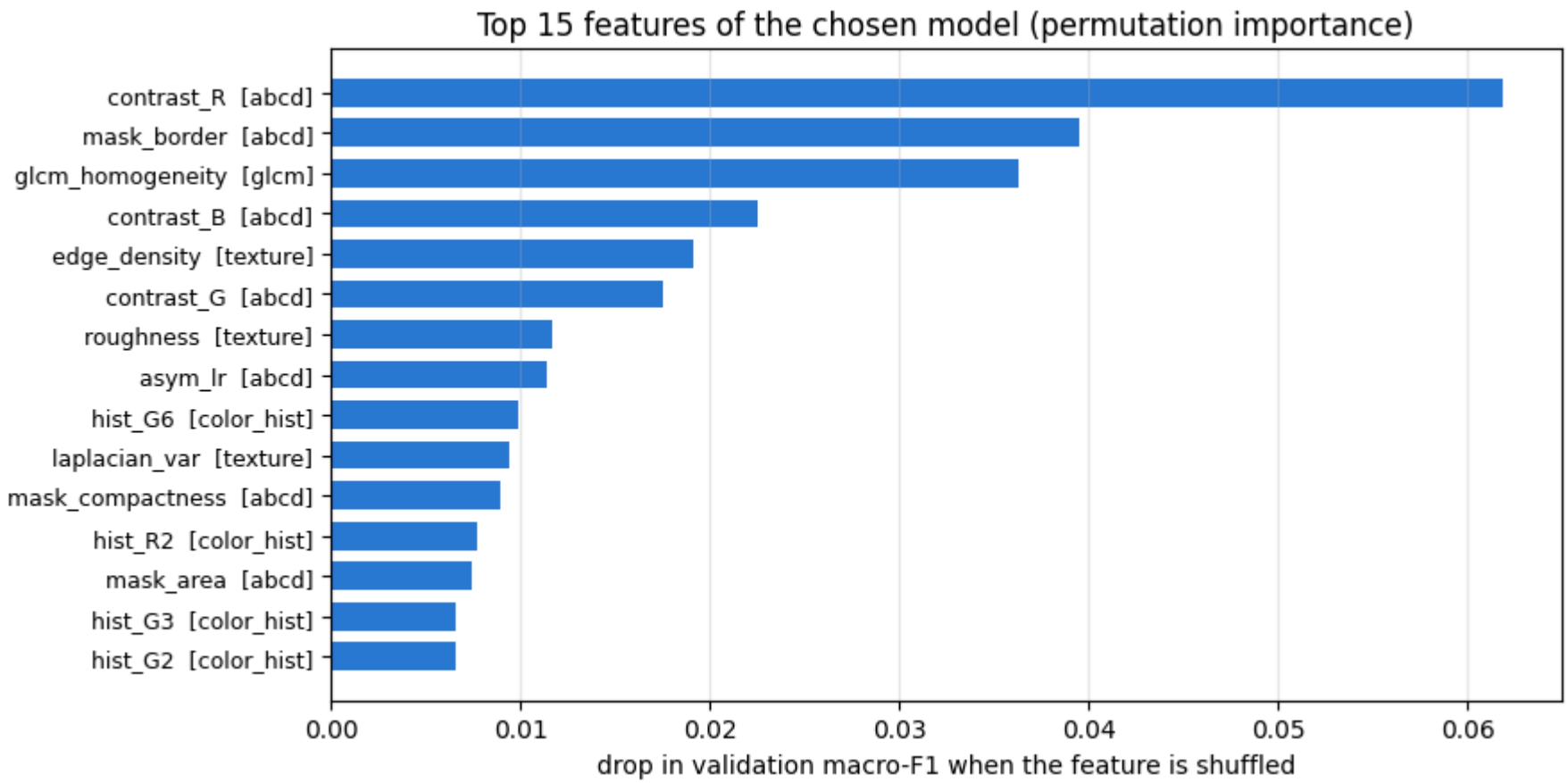
3.3 Which features does the chosen model use?

Permutation importance on VALIDATION: shuffle one feature at a time and measure how much the validation macro-F1 drops. Works the same way for every model type.

In [12]:
```
pi = permutation_importance(model, X_va, y_va, scoring="f1_macro", n_repeats=5, random_state=config.SEED, n_jobs=-1)
imp = pd.Series(pi.importances_mean, index=cols).sort_values(ascending=False)
groups = features.feature_groups(cols)
group_of = {c: g for g, cs in groups.items() for c in cs}

fig, ax = plt.subplots(figsize=(9, 4.6))
top15 = imp.head(15)[::-1]
ax.barh(range(len(top15)), top15.values, color="#2a78d6", height=0.7)
ax.set_yticks(range(len(top15)), [f"{c}  [{group_of[c]}]" for c in top15.index], fontsize=9)
ax.set_xlabel("drop in validation macro-F1 when the feature is shuffled")
ax.set_title("Top 15 features of the chosen model (permutation importance)")
ax.grid(axis="x", alpha=0.3); plt.tight_layout(); save("feature_importance"); plt.show()

print("importance summed per feature group:")
imp.groupby(imp.index.map(group_of)).sum().sort_values(ascending=False).round(4)
```




```
importance summed per feature group:
```

Out[12]:
```
abcd           0.1670
color_hist     0.0850
texture        0.0403
glcm           0.0395
color_stats    0.0012
dtype: float64
```

Observation. The model relies most on the **ABCD features**: the skin–lesion colour contrast (contrast_R/B/G), how much of the image border the lesion covers (mask_border), and asymmetry. Next come GLCM homogeneity and edge density. *Colour statistics ≈ 0* does not mean colour is useless: the same information is also in the histograms and the contrasts. Permutation importance gives correlated features low scores, because shuffling one of them leaves its copies intact.

3.4 A clinical operating point (chosen on VALIDATION)

The default rule (malignant if the score passes the model's default threshold) weighs both errors the same. A missed cancer is worse than an unnecessary check, so I also fix a second threshold **on validation** that catches **≥ 90 % of malignant lesions**, and report what it costs in specificity.

In [13]:
```
thr90 = classical.threshold_for_sensitivity(y_va, score_va, target=0.90)
default_pred_va = model.predict(X_va)
if best["sensitivity"] >= 0.90:
    # the default rule already catches >= 90%: keep it (never make the model LESS sensitive)
    thr90 = min(thr90, score_va[default_pred_va == 1].min())
    print("default threshold already reaches >= 90% sensitivity on validation -> it is kept")
pred_va90 = (score_va >= thr90).astype(int)
m90 = classical.binary_metrics(y_va, pred_va90, score_va)
print(f"threshold for >=90% sensitivity (validation): {thr90:.4f}")
pd.DataFrame({"default threshold": {k: best[k] for k in m90}, "sensitivity-90 threshold": m90}).round(3)
```

```
threshold for >=90% sensitivity (validation): -0.2288
```

Out[13]:

	default threshold	sensitivity-90 threshold
macro_f1	0.741	0.753
balanced_acc	0.743	0.741
accuracy	0.788	0.810
sensitivity	0.847	0.900
specificity	0.639	0.581
roc_auc	0.797	0.797

Observation. At the default threshold the model catches 84.7 % of malignant lesions on validation. Lowering the threshold (score ≥ −0.229) catches 90 %, at the cost of specificity dropping from 63.9 % to 58.1 %.

---

4. TEST — evaluated once

Everything above (feature set, model, hyper-parameters, threshold) was decided on validation only. The test split is used here for the first and only time, with the chosen configuration trained on TRAIN.

In [14]:
```
X_te, y_te = feats.loc[test.isic_id, cols].to_numpy(), test["label"].to_numpy()
pred_te = model.predict(X_te)
score_te = classical.scores_of(model, X_te)
pred_te90 = (score_te >= thr90).astype(int)

test_default = classical.binary_metrics(y_te, pred_te, score_te)
test_90 = classical.binary_metrics(y_te, pred_te90, score_te)
ci = classical.bootstrap_ci(y_te, pred_te, score_te)
majority_test = classical.binary_metrics(y_te, np.ones_like(y_te) * int(train["label"].mode()[0]))

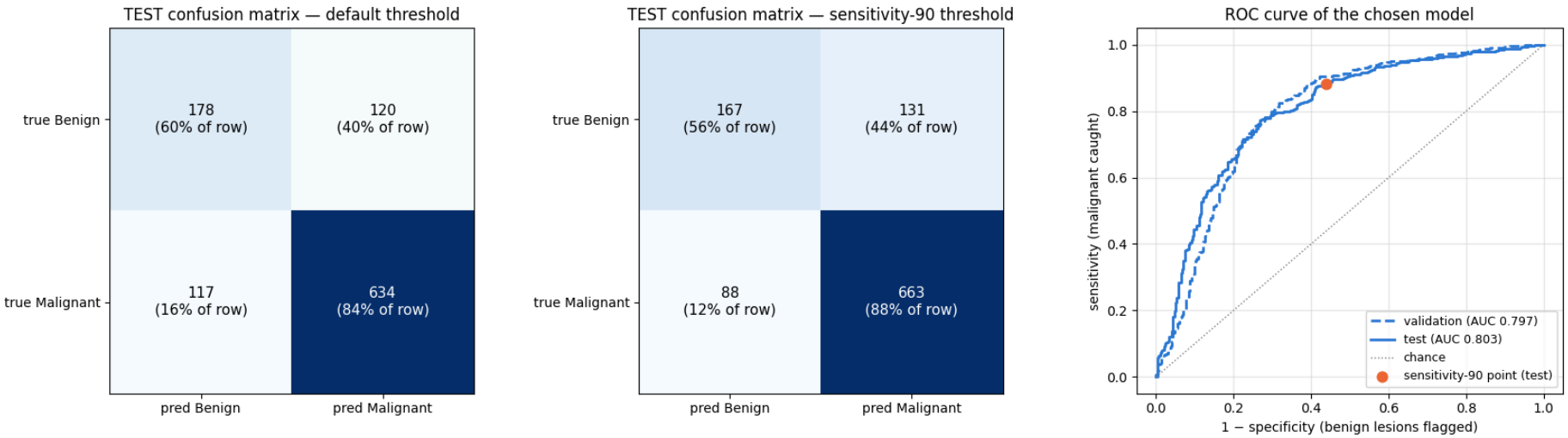
summary = pd.DataFrame({
    "majority (floor)": majority_test,
    "chosen model — validation": {k: best[k] for k in test_default},
    "chosen model — TEST": test_default,
    "TEST 95% CI": {k: f"[{ci.loc[k, 'low']:.3f}, {ci.loc[k, 'high']:.3f}]" for k in test_default},
    "TEST, sensitivity-90 threshold": test_90,
})
summary.round(3)
```

Out[14]:

	majority (floor)	chosen model — validation	chosen model — TEST	TEST 95% CI	TEST, sensitivity-90 threshold
macro_f1	0.417	0.741	0.721	[0.691, 0.751]	0.731
balanced_acc	0.500	0.743	0.721	[0.688, 0.751]	0.722
accuracy	0.716	0.788	0.774	[0.749, 0.799]	0.791
sensitivity	1.000	0.847	0.844	[0.816, 0.870]	0.883
specificity	0.000	0.639	0.597	[0.539, 0.652]	0.560
roc_auc	NaN	0.797	0.803	[0.772, 0.834]	0.803

In [15]:
```
fig, axes = plt.subplots(1, 3, figsize=(17, 4.8))
for ax, (title, pred) in zip(axes[:2], [("default threshold", pred_te), ("sensitivity-90 threshold", pred_te90)]):
    cm = classical.confusion_table(y_te, pred, BINARY)
    ax.imshow(cm.values, cmap="Blues")
    for i in range(2):
        for j in range(2):
            v = cm.values[i, j]
            ax.text(j, i, f"{v}\n({v / cm.values[i].sum():.0%} of row)", ha="center", va="center", fontsize=11,
                    color="white" if v > cm.values.max() / 2 else "black")
    ax.set_xticks([0, 1], ["pred Benign", "pred Malignant"]); ax.set_yticks([0, 1], ["true Benign", "true Malignant"])
    ax.set_title(f"TEST confusion matrix — {title}")

ax = axes[2]
for name, y, s, style in [("validation", y_va, score_va, "--"), ("test", y_te, score_te, "-")]:
    fpr, tpr, _ = roc_curve(y, s)
    ax.plot(fpr, tpr, style, color="#2a78d6", lw=2, label=f"{name} (AUC {roc_auc_score(y, s):.3f})")
ax.plot([0, 1], [0, 1], ":", color="gray", lw=1, label="chance")
ax.scatter(1 - test_90["specificity"], test_90["sensitivity"], s=60, color="#eb6834", zorder=3,
           label="sensitivity-90 point (test)")
ax.set_xlabel("1 − specificity (benign lesions flagged)"); ax.set_ylabel("sensitivity (malignant caught)")
ax.set_title("ROC curve of the chosen model"); ax.legend(fontsize=9); ax.grid(alpha=0.3)
plt.tight_layout(); save("test_confusion_roc"); plt.show()
```



Observation (TEST).

- **macro-F1 0.721** [95 % CI 0.691–0.751] and **ROC-AUC 0.803**, against 0.417 for the majority class.
- The drop from validation (0.741) is within the confidence interval. It is the expected small optimism of choosing the best of 105 configurations on validation.
- At the default threshold the model misses **117 of 751 malignant lesions (16 %)** and flags **40 % of benign lesions**.
- The sensitivity-90 threshold reaches 88.3 % sensitivity on test, slightly below the 90 % target because a threshold never transfers exactly to new data. It misses 29 fewer cancers (88 instead of 117) for 11 more false alarms (131 instead of 120).

4.1 Which diagnoses are hard? (error analysis on TEST)

Share of correctly classified test lesions in each of the 11 diagnoses (default threshold).

In [16]:
```
per_dx = classical.per_group_recall(y_te, pred_te, test["dx"])
per_dx["true class"] = per_dx["label"].map({0: "Benign", 1: "Malignant"})
per_dx = per_dx.drop(columns="label").rename(columns={"correct": "share correct"})
per_dx.to_csv(BASE_DIR / "test_per_diagnosis.csv")
per_dx.round(3)
```

Out[16]:

group	n	share correct	true class
BKL	109	0.248	Benign
MEL	90	0.344	Malignant
DF	11	0.364	Benign
BEN_OTH	9	0.444	Benign
INF	10	0.500	Benign
MAL_OTH	2	0.500	Malignant
VASC	10	0.700	Benign
NV	149	0.879	Benign
SCCKA	94	0.894	Malignant
BCC	504	0.917	Malignant
AKIEC	61	0.918	Malignant

Observation — the most important finding. The errors are not spread evenly:

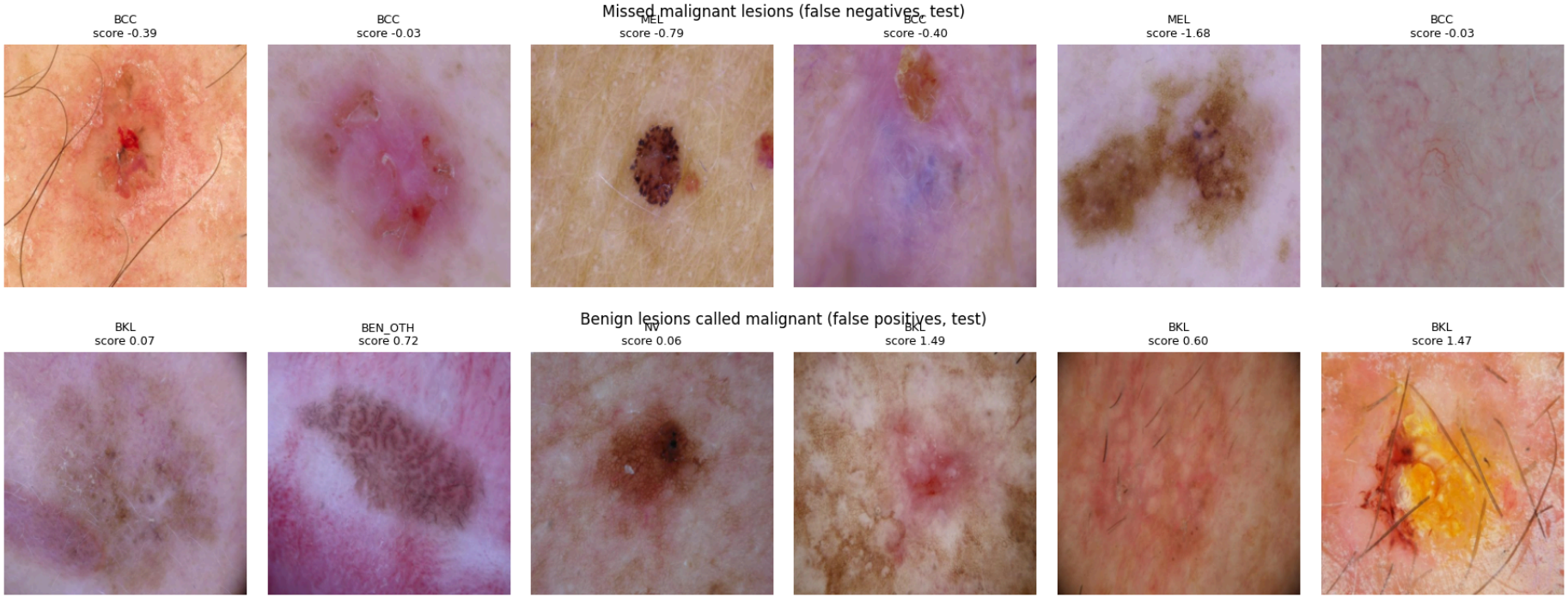
- **Melanoma (MEL): only 34 % caught.** BCC, AKIEC and SCCKA: about 90 % caught.
- **Benign keratosis (BKL): only 25 % correct.** NV: 88 % correct.

The model learned the dominant pattern of the data: *dark, high-contrast, pigmented = benign (like NV)* and *pink, scaly, low-contrast = malignant (like BCC)*. Melanomas are pigmented, so they look "benign" to it. Benign keratoses are often scaly and irregular, so they look "malignant". The macro-F1 hides this, and clinically it is the worst possible error, because melanoma is the most dangerous of these cancers.

In [17]:
```
def show_examples(mask, title, name, n=6):
    rows = test[mask].sample(min(n, mask.sum()), random_state=config.SEED)
    fig, axes = plt.subplots(1, n, figsize=(2.9 * n, 3.3))
    for ax in axes: ax.axis("off")
    for ax, r in zip(axes, rows.itertuples()):
```

```
        ax.imshow(features.load_for_features(image_paths[r.isic_id]))
        ax.set_title(f"{r.dx}\nscore {score_te[r.Index]:.2f}", fontsize=9)
    plt.suptitle(title); plt.tight_layout(); save(name); plt.show()

show_examples((y_te == 1) & (pred_te == 0), "Missed malignant lesions (false negatives, test)", "false_negatives")
show_examples((y_te == 0) & (pred_te == 1), "Benign lesions called malignant (false positives, test)", "false_positives")
```



Observation. The missed malignant lesions include pigmented melanomas and very faint pink BCCs. The false alarms are mostly BKL lesions with scales, redness or a yellow crust. Hairs are visible in many images and add edges that have nothing to do with the lesion (Session 5 warning).

---

5. Extra: the 3-class task (Benign / Malignant / Indeterminate)

Same features, models and grids, but with the 3 original `diagnosis_1` classes, as in the Session 5 slides. Again: selection on validation with macro-F1, then test once.

In [18]:
```
results3 = classical.run_grid(feats, train, val, "label3", THREE)
results3.assign(params=results3["params"].astype(str)).to_csv(BASE_DIR / "val_results_3class.csv", index=False)
cells3 = classical.best_per_cell(results3)
grid3 = cells3.pivot(index="feature_set", columns="model", values="macro_f1").loc[set_order, model_order]
grid3.round(3)
```

```
color (30)               majority       best val macro-F1 = 0.272
color (30)               logistic_reg   best val macro-F1 = 0.425
color (30)               knn            best val macro-F1 = 0.443
color (30)               random_forest  best val macro-F1 = 0.520
color (30)               svm_rbf        best val macro-F1 = 0.512
texture (3)              majority       best val macro-F1 = 0.272
texture (3)              logistic_reg   best val macro-F1 = 0.297
texture (3)              knn            best val macro-F1 = 0.399
texture (3)              random_forest  best val macro-F1 = 0.416
texture (3)              svm_rbf        best val macro-F1 = 0.376
color+texture (33)       majority       best val macro-F1 = 0.272
color+texture (33)       logistic_reg   best val macro-F1 = 0.464
color+texture (33)       knn            best val macro-F1 = 0.448
color+texture (33)       random_forest  best val macro-F1 = 0.553
color+texture (33)       svm_rbf        best val macro-F1 = 0.548
color+texture+GLCM (37)  majority       best val macro-F1 = 0.272
color+texture+GLCM (37)  logistic_reg   best val macro-F1 = 0.467
color+texture+GLCM (37)  knn            best val macro-F1 = 0.488
color+texture+GLCM (37)  random_forest  best val macro-F1 = 0.534
color+texture+GLCM (37)  svm_rbf        best val macro-F1 = 0.538
all incl. ABCD (45)      majority       best val macro-F1 = 0.272
all incl. ABCD (45)      logistic_reg   best val macro-F1 = 0.478
all incl. ABCD (45)      knn            best val macro-F1 = 0.494
all incl. ABCD (45)      random_forest  best val macro-F1 = 0.538
all incl. ABCD (45)      svm_rbf        best val macro-F1 = 0.567
```

Out[18]:

model	majority	logistic_reg	knn	random_forest	svm_rbf
feature_set					
color (30)	0.272	0.425	0.443	0.520	0.512
texture (3)	0.272	0.297	0.399	0.416	0.376
color+texture (33)	0.272	0.464	0.448	0.553	0.548
color+texture+GLCM (37)	0.272	0.467	0.488	0.534	0.538
all incl. ABCD (45)	0.272	0.478	0.494	0.538	0.567

In [19]:

```
best3 = classical.select_best(results3)
print("CHOSEN (3-class):", best3["feature_set"], "|", best3["model"], "|", best3["params"])
model3, cols3 = classical.fit_config(best3, feats, train, "label3")
y3_te = test["label3"].to_numpy()
pred3_te = model3.predict(feats.loc[test.isic_id, cols3].to_numpy())
test3 = classical.multiclass_metrics(y3_te, pred3_te, THREE)
floor3 = classical.multiclass_metrics(y3_te, np.zeros_like(y3_te) + int(train["label3"].mode()[0]), THREE)
display(pd.DataFrame({"majority (floor)": floor3,
                      "chosen — validation": {k: best3[k] for k in test3},
                      "chosen — TEST": test3}).round(3))
classical.confusion_table(y3_te, pred3_te, THREE)
```

```
CHOSEN (3-class): all incl. ABCD (45) | svm_rbf | {'C': 3}
```

	majority (floor)	chosen — validation	chosen — TEST
macro_f1	0.273	0.567	0.518
balanced_acc	0.333	0.647	0.582
accuracy	0.694	0.728	0.704
recall_Benign	0.000	0.628	0.594
recall_Malignant	1.000	0.776	0.760
recall_Indeterminate	0.000	0.536	0.391

Out[19]:

	pred Benign	pred Malignant	pred Indeterminate
true Benign	177	105	16
true Malignant	111	553	64
true Indeterminate	1	13	9

Observation (3 classes). The same configuration wins again (all 45 features + SVM, C = 3). On test it reaches **macro-F1 0.518**, against a majority floor of 0.273 (the slides' 0.27). It finds 9 of the 23 *Indeterminate* lesions (39 %). With only 23 such lesions in test this number is very uncertain, and many malignant lesions are also pushed into Indeterminate (64). This supports the Milestone 1 decision to merge Indeterminate into Malignant for the main task.

6. Save the key numbers

In [20]:
```
out = {
    "binary": {
        "chosen": {"feature_set": best["feature_set"], "model": best["model"], "params": best["params"]},
        "validation": {k: float(best[k]) for k in test_default},
        "test_default_threshold": {k: float(v) for k, v in test_default.items()},
        "test_95ci": {k: [float(ci.loc[k, "low"]), float(ci.loc[k, "high"])] for k in test_default},
        "sensitivity90_threshold": float(thr90),
        "test_sensitivity90_threshold": {k: float(v) for k, v in test_90.items()},
        "test_majority": {k: float(v) for k, v in majority_test.items() if not np.isnan(v)},
    },
    "three_class": {
        "chosen": {"feature_set": best3["feature_set"], "model": best3["model"], "params": best3["params"]},
        "validation": {k: float(best3[k]) for k in test3},
        "test": {k: float(v) for k, v in test3.items()},
        "test_majority": {k: float(v) for k, v in floor3.items()},
    },
    "n_lesions": {name: len(df) for name, df in splits.items()},
    "seed": config.SEED,
}
with open(BASE_DIR / "test_results.json", "w") as f:
    json.dump(out, f, indent=2)
print("saved:", sorted(p.name for p in BASE_DIR.iterdir()))
```

```
saved: ['features_dermoscopic.csv', 'test_per_diagnosis.csv', 'test_results.json', 'val_results_3class.csv', 'val_results_binar
y.csv']
```

7. Report

7.1 Summary

A classical baseline (handcrafted features + scikit-learn models) was developed for the project task, **Benign vs Malignant** from the dermoscopic image, on the Milestone 1 lesion-level split (3,143 / 1,048 / 1,049 lesions). **5 feature sets × 5 models × small hyper-parameter grids = 105 configurations** were compared on validation with macro-F1. The chosen configuration was then evaluated once on test.

	validation	test	test 95 % CI
majority class (floor) — macro-F1	0.418	0.417	—
chosen: all 45 features + SVM (RBF, C = 3) — macro-F1	0.741	**0.721**	0.691 – 0.751
balanced accuracy	0.743	0.721	0.688 – 0.751
ROC-AUC	0.797	0.803	0.772 – 0.834
sensitivity (malignant caught)	0.847	0.844	0.816 – 0.870
specificity (benign correctly left alone)	0.639	0.597	0.539 – 0.652
sensitivity-90 threshold: sensitivity / specificity	0.900 / 0.581	0.883 / 0.560	
3-class extra — macro-F1 (floor 0.273)	0.567	0.518	

The baseline number for later milestones: test macro-F1 0.72, ROC-AUC 0.80. The CNN of Session 7 has to beat this, on the same split and with the same metric.

7.2 Discussion

1. **The task is learnable from simple signals.** Every model clearly beats the majority floor. Colour carries most of the information. The Session 4 texture features help only a little on top of colour, and GLCM only helps the linear model. The dermatology-inspired ABCD features (skin–lesion contrast, border, asymmetry) give the best results.

2. **Model choice matters less than feature choice.** SVM ≈ random forest > logistic regression > kNN, but the spread between models (about 0.04) is smaller than between feature sets (up to 0.14). The top configurations are statistically tied on validation.
3. **Honest evaluation.** All decisions were made on validation. Test was used once, and its score is close to validation (0.721 vs 0.741, inside the 95 % interval), so the selection overfitted only slightly.
4. **Accuracy would have been misleading.** The majority class gets 71.6 % test accuracy and learns nothing; the chosen model reaches only 77.4 % accuracy. Macro-F1 (0.42 → 0.72) and balanced accuracy (0.50 → 0.72) show the real gain.
5. **The model learned the class mix, not malignancy.** Because 67 % of malignant lesions are BCC and most benign ones are nevi, it treats *pigmented and high-contrast* as benign. Consequences on test: **only 34 % of melanomas are caught**, and **75 % of benign keratoses are called malignant**. One overall score hides this. A deployable tool would have to be evaluated per diagnosis, with special attention to melanoma.
6. **Clinical trade-off.** Lowering the threshold (fixed on validation) catches more cancers (88 instead of 117 missed) at the cost of a few more false alarms. For a triage tool, where a missed cancer costs more than an extra check, this is the preferable operating point.

7.3 Limitations

- **Whole-image features:** colour and texture are measured on the whole image, so skin, hairs and the dark dermoscope corners are mixed in with the lesion.
- **Rough mask:** the mask is a simple threshold. It fails on 13 % of images (pale, non-pigmented lesions) and sometimes cuts off part of the lesion.
- **Hairs:** hairs are strong edges and confuse the texture features (no hair removal was applied).
- **Resizing:** every image is resized to 256 × 256, which distorts the aspect ratio. Size features are in pixels, not mm (no scale available).
- **One validation split:** selection used a single validation split, not cross-validation, and the top configurations are within the noise of each other.
- **Rare classes:** small classes (DF, INF, VASC, BEN_OTH, MAL_OTH and the 3-class *Indeterminate*) have too few test lesions for reliable per-class numbers.
- **One image per lesion:** only the dermoscopic image is used. The clinical close-up and the metadata (age, site) are not used.

7.4 Next steps

- **Session 7 — CNN:** compare it with this baseline on the same split and metrics, and **report recall per diagnosis** (especially MEL).

- **Session 8 — segmentation:** a proper lesion mask would allow true border, asymmetry and diameter features, and colour measured only inside the lesion.
- **Cheap improvements to the classical model:** hair removal (morphological black-hat filter), colour normalisation (colour constancy), and more texture descriptors (LBP, Gabor filters, HSV/Lab colour). Use cross-validation on train + validation for the model selection.